

20/08/26

Agentic AI Frameworks

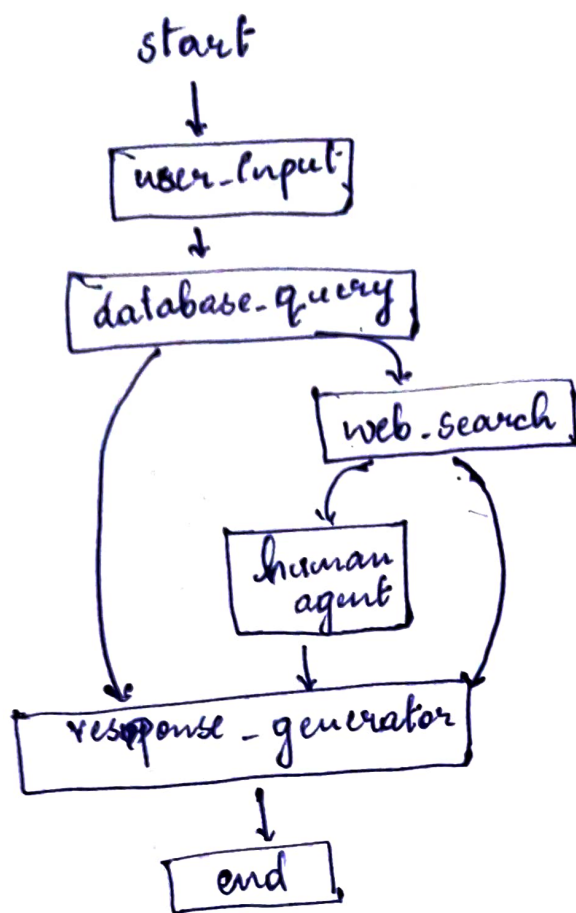
* LangGraph -

→ open-source framework for building LLM-based applications.

→ It helps to build agents that can think, decide, use tools/APIs, take actions.

- # Node - a task / tool / action
 - # Edge - connection between nodes
 - # State - information of nodes
- } components of LangGraph

State management -- remember details from previous activities.



It is designed for complex workflows -

1. Branching Logic: workflow $\leftrightarrow$ different branches.

2. Fallback mechanisms: backup plan

3. Multi-step decisions: multiple steps $\rightarrow$ each step depends on the outcome of previous one.

$\rightarrow$ State machine is also a component of LangGraph.

example syntax of state:

state = { [] } # previous one

state = { "messages": [

{ "role": "user", "content": "Hello. What is AI?" },

{ "role": "db-query-tool",

}] "content": "Hi, I am not sure abt the answer." }

- State machine decides the states; what next node should be connected.
 - State machine takes helps of the edges.
- example: if db-query = True then directly go to response-generate, else to web-search.

Practical

from typing import List, Dict, ^{storing state} state machine
 from langgraph.graph import StateGraph, START, END
 from langchain-ollama.llms import OllamaLLM

Step 1: Define state

class State(Dict):

messages: List[Dict[str, str]]

Step 2: Initialize StateGraph (state machine)

graph_builder = StateGraph(State)

initialize LLM

llm = OllamaLLM(model = "llama3.1")

define chatbot function

def chatbot(state: State):

response = llm.invoke(state["messages"])

state["messages"].append({ "role": "assistant",
 "content": response })

```
return {"messages": state["messages"]}
```

Add nodes and edges

```
graph_builder.add_node("chatbot", chatbot)
"      .add_edge(START, "chatbot") # starting node
"      ("chatbot", END)
```

Compile the graph

```
graph = graph_builder.compile()
```

Stream updates

```
def stream_graph_updates(user_input: str):
    # initialize state with the user's input
    state = {"messages": [{ "role": "user", "content":
                                user_input }]}

    for event in graph.stream(state):
        for value in event.values():
            print("Assistant", value["messages"][-1]
                  ["content"])
```

Run chatbot in a loop

```
if __name__ == "__main__":
    while True:
        try:
            user_input = input("User: ")
            if user_input.lower() in ["quit", "exit", "q"]:
                print("Goodbye!")
                break
            stream_graph_updates(user_input)
```

except Exception as e:

print(f"Error occurred: {e}")

break

How to create agents in LangGraph?

- LLM : understands user query & generates responses

- tools : they are external functionalities.

If LLM doesn't know the answer, it can use tools to answer the question.

Practical

→ from langchain-community import WikipediaQueryRun
" .utilities import WikipediaAPIWrapper

api-wrapper = WikipediaAPIWrapper(top-k-results=1, ^{± result with 100 characters}
doc-content-chars-max=100)

wiki-tool = WikipediaQueryRun(api-wrapper=api-wrapper)

→ wiki-tool.run({"query": "AI agents"})

→ from langchain-openai import ChatOpenAI

llm = ChatOpenAI(temperature=0, api-key="", model=" ")

→ tools = [wiki-tool]

tools binding

llm-with-tools = llm.bind_tools(tools)

tool calling

result = llm-with-tools.invoke("Hello World")

result

result.content

We'll create the agent

→ from langgraph.prebuilt import create_react_agent
agent_executor = create_react_agent(llm, tools)

→ from langchain_core.messages import HumanMessage

this is simple query so AI answers it.

response = agent_executor.invoke({"messages":

[HumanMessage(content="hi!")]})

response["messages"]

So in o/p we'll have

AI Message (content = 'Hello! How can I assist you?')

You view only message 1,

→ print(response["messages"][-1].content)

When we change Hi → what is agent AI?
then o/p will be

ToolMessage (content = 'Page : Intelligent agent In

Summary : _____, tool-call-id = ' ')

24/08/26

→ use tools?

→ create custom tools?

→ multiple tools with agents?

Custom Tools

```
→ from IPython.display import Image, display
    " langgraph.graph import StateGraph, START
    " langchain-openai ~ ChatOpenAI
    import requests
    from langchain-core.messages import SystemMessage
    HumanMessage
    from langgraph.graph import MessageState
    " , prebuilt import ToolNode, tools_condition
    from typing import Annotated
    " -extensions import TypedDict
    from langgraph.graph.message import add_messages
```

```
→ class State(TypedDict):
    messages: Annotatedto [list, add_messages]
```

```
→ !pip install -U duckduckgo-search ↴
                                     web search tool
```

```
→ from langchain-community.tools import
    DuckDuckGoSearchRun
```

```
{ search = DuckDuckGoSearchRun()
  { search.invoke("Obama's first name?")
  in function }
```

```
def search-duckduckgo(query: str):
```

```
    """ Searches DuckDuckGo using langchain's
    DuckDuckGoSearchRun tool. """
```

```
    search = D ~ Run()
```

```
return search.invoke(query)
```

```
result = search - duckduckgo ("What is AI agent")  
print (result)
```

**** Tools are functions in LangGraph.**

→ # llm import and initialize

→ llm.invoke('hello').content

Now we'll use the functions like add, multiply.

```
def add (a: int, b: int) → int:
```

```
    return a + b    # Similarly multiply
```

→ tools = [search-duckduckgo, add, multiply]

```
llm-with-tools = llm.bind_tools (tools)
```

→ def chatbot (state; State):

```
    return { "messages": [llm-with-tools.invoke  
                           (state ["messages"])]  
            }
```

→ from langgraph.prebuilt import ToolNode,
tools-condition

```
graph-builder = StateGraph (State)
```

nodes

```
graph-builder.add_node ("assistant", chatbot)
```

```
    ("tools", ToolNode (tools))
```

define edges

```
graph-builder.add_edge (START, "assistant")
```

if assistant = llm doesn't have the answer then
it checks with tools.

```
graph_builder.add_conditional_edges("assistant",  
                                     tools_condition)
```

```
" .add_edge("tools", "assistant")
```

```
react_graph = graph_builder.compile()
```

→ # visually

```
display(Image(react_graph.get_graph().
```

```
draw_mermaid_png()))
```

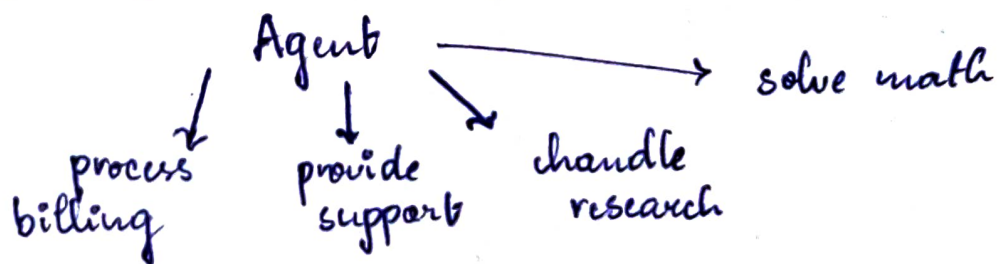
→ response = react_graph.invoke({ "messages":

```
[HumanMessage(content = "what is Delhi's weather.  
Multiply it by 2  
and add 5. ")]})
```

```
print(response["messages"])
```

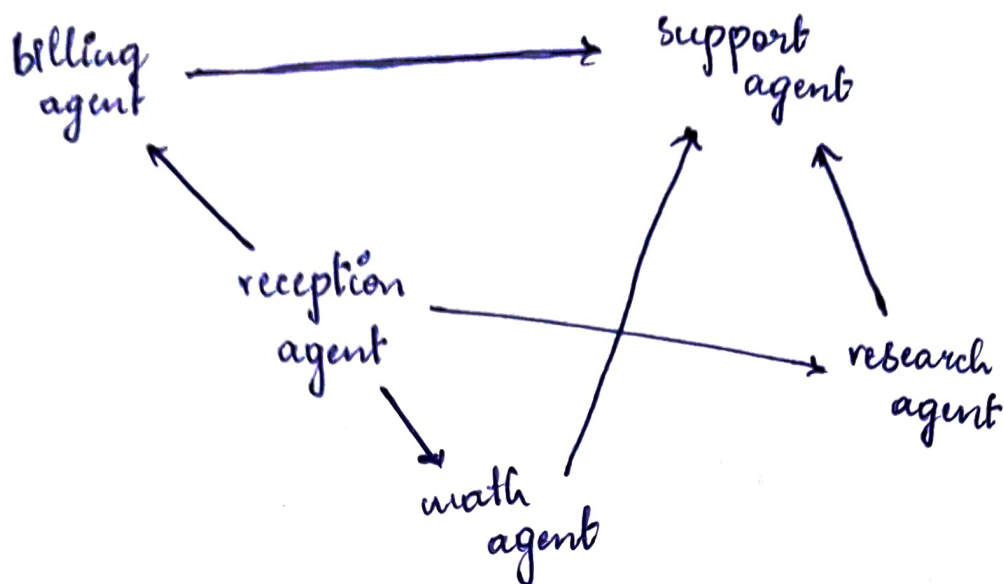
* Multi-Agent System

→ single-agent system handling all tasks.



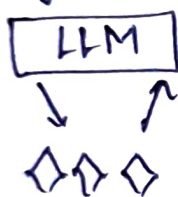
Can sometimes pick wrong path.

→ Multi-agent system

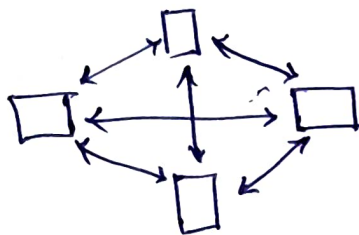


Different ways agent can interact :

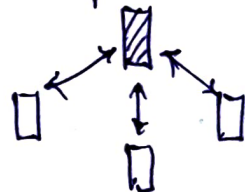
single agent



Network



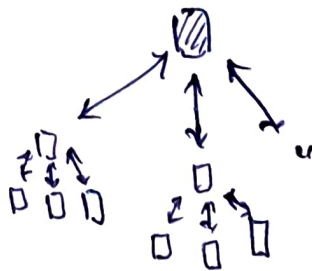
Supervisor →



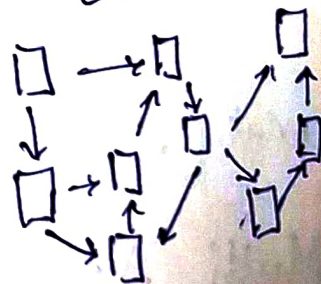
Supervisor
(as tools)



hierarchical



Custom



→ Multi-agent systems can be considered visually as graphs.

visually as graphs.

Each agent performs a task and hands off control to another agent.



```
graph TD; A[ ] --> B[Doctor];
```

example : Receptionist $\rightarrow$ Nurse $\rightarrow$ Doctor

sample code example:

```
def s_agent(state):
```

```
    if "math" in state["task"]:
```

```
        return Command(
```

```
            goto = "math-agent",
```

hand off
to any
agent

```
            update = {"info": "Passing to Math  
Expert"}  
        )
```

```
    else:
```

```
        return "Research completed!"
```

* Supervisor Way

```
→ from langchain-openai import ChatOpenAI
```

```
    " langgraph-supervisor " create-supervisor
```

```
    " " . prebuilt import create-react-agent
```

```
model = ChatOpenAI (temperature=0, api_key = " ", model=" ")
```

```
→ install DuckDuckGoSearchRun
```

```
→ initialize it in a function
```

```
    add , multiply functions
```

```
→ # agent creation
```

```
math_agent = create-react-agent ( model=model,
```

```
    tools = [add, multiply],
```

```
    name = "math-expert",
```

```
    prompt = "You are a math ~"
```

```
)
```

→ # other agent - same syntax

```
research_agent = Agent(
    tools = [web_search],
    name = "research-expert",
    " = " "
)
```

→ # create supervisor workflow

```
workflow = create_supervisor(model=model,
    [research_agent, math_agent],
```

```
prompt = ("You are a team —"
```

```
" For current events, use
    research-agent."
```

```
) " For math, use math-agent"
```

```
)
```

→ # Compile and run

```
app = workflow.compile()
```

```
result = app.invoke({
```

```
"messages": [{ "role": "user",
```

```
"content": "what is AI?" }]
```

```
})
```

→ result

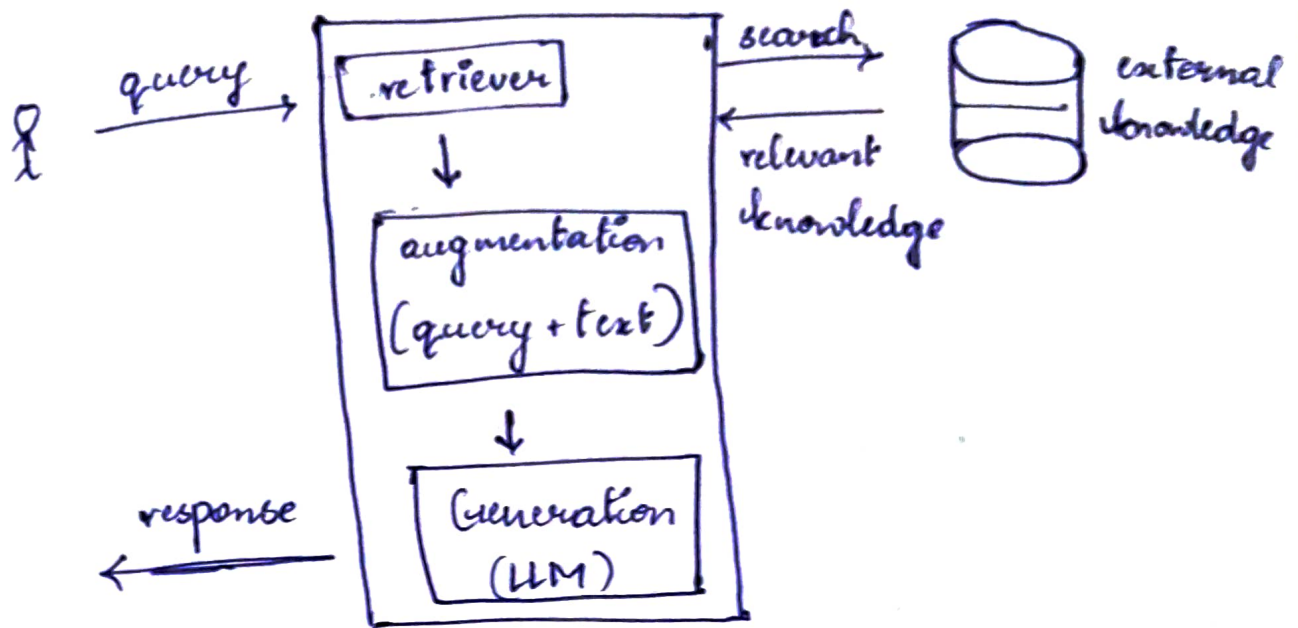
→ # viewing with confusion

```
for m in result["messages"]:
```

```
m.pretty_print()
```

shows the transfer calls along with answers.

* Retrieval-Augmented Generation (RAG)



Why RAG?

In simple terms traditional LLM trained in 2024 could not (may not) answer updated news in 2028.

→ can't access new information.

→ models don't work well with private data.

Practical

→ from `langchain-community`. `document_loaders`
`import UnstructuredURLLoader`

```
urls = [ ' ', ' ' ]
```

```
loader = UnstructuredURLLoader(urls=urls)
```

```
docs = loader.load()
```

→ docs

We can't push whole data so, chunks.

→ from langchain-text-splitters import

text_splitter = RecursiveCharacterTextSplitter
" (chunk_size=1000, chunk_overlap=200)

all_splits = text_splitter.split_documents(docs)

print("Total no. of docs:", len(all_splits))

*** Overlap (chunks) is used to review previous chunk information. ***

Embedding model: text - numbers.

→ from langchain-community.embeddings import

embeddings = HuggingFaceEmbeddings
"()

Vector Database - Chroma

→ from langchain-chroma import Chroma

"-core.documents import Document

vector_store = Chroma.from_documents(
documents = all_splits,

embedding = H-F-Embeddings())

for storing locally include

(documents=docs, embedding="()",
persist_directory=". / my-chroma-db")

→ # import required

model_id = "tiiuae/falcon-7b"

text-generation-pipeline = pipeline(

 "text-generation",

 model=model_id,

 model_kwargs={"torch_dtype": torch.bfloat16},

 max_new_tokens=200,

 device=0,

 temperature=0.7,

 top_k=50, # filters low-probability tokens

)

llm = HuggingFacePipeline(pipeline=text-generation-pipeline)

→ from langchain_core.prompts import PromptTemplate

template = """

 {context}

 Question: {question}

 Helpful answer: ""

prompt = PromptTemplate.from_template(template)

ready made prompt

from langchain import hub

prompt = hub.pull("rlm/rag-prompt")

→ from typing_extensions import List, TypedDict

class State (TypedDict):

question: str

content: List[Document]

answer: str

→ # Define application steps

def retrieve (state: State):

retrieved_docs = vectordb.similarity_search(
state["question"], k=1)

return {"context": retrieved_docs}

→ def generate (state: State):

docs_content = "\n\n".join (doc.page_content for doc
in state["content"])

messages = prompt.invoke ({ "question": state["question"],
"context": docs_content })

response = llm.invoke (messages)

return {"answer": response}

We need a state machine

from langgraph.graph import START, StateGraph

Compile application and test

graph_builder = StateGraph (State).add_sequence
([retrieve, generate])

.add_edge (START, "retrieve")

graph = graph_builder.compile()

testing

```
→ response = graph.invoke({"question": "what is CP?"})  
print(response["answer"])
```